

DATA 260 - Agentic AI

Homework 6 - Evidence Report

Student Name: Shoury Parab

SID4: 7085

Repository: data260-7085

Branch: hw6

Tagged commit: 501d41316a73d5513956bed38265a5333324d509

Hardware: Intel Core i5-6200U @ 2.30GHz

Configuration: PORT_BASE=8785, PREFIX=s7085, SEED=7085, VERIFY_SEED=267085,

DOMAIN_ID=5

Local model: Ollama llama3.2:3b

Repository: <https://github.com/shoddy16/data260-7085>

Submission status

This report was generated from the current HW6 branch and checked-in machine-readable results. FastAPI, PyMongo, and Ollama dependencies are installed and the API validation smoke checks pass. Docker Desktop's MongoDB engine is unavailable, so live MongoDB collection evidence and dependent screenshots are not claimed.

Part 1 - MongoDB Document API

The implementation adds a MongoDB-backed document view at `/api/documents` with required short text, optional description, constrained status/category fields, an inspection datetime, automatic timestamps, Pydantic validation, and MongoDB JSON Schema validation. CRUD handlers return explicit validation, not-found, database, and success status codes.

Relevant implementation: `hw6_mongo.py`, `hw6_memory.py`, `routers/hw6.py`, and `main.py`. Live MongoDB screenshots remain pending because MongoDB and PyMongo were unavailable in this run.

Part 2 - Short-, Long-, and Episodic Memory

The `/api/chat`, `/api/memory/{user_id}`, and `/api/aggregate/{user_id}` routes use messages, summaries, and episodes. Chat turns save user and assistant messages, assemble system/session/lifetime/short-term/episodic context, create deterministic local embeddings, retrieve episodes by cosine similarity, and trigger summaries every configured number of user messages.

Offline deterministic tests and FastAPI route validation pass. A live Ollama session was attempted, but the ten-turn memory run exceeded the bounded execution window; MongoDB Compass, persisted collection, memory JSON, aggregate JSON, and screenshot evidence remain blocked by the unavailable MongoDB service.

Part 3 - What Memory Costs

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality
SHORT_TERM_N=4	299.733	0.007	3.5/5
SHORT_TERM_N=12	412.2	0.002	4.0/5

SHORT_TERM_N=30	601.267	0.007	4.0/5
compression_policy	296.733	0.007	3.9/5

The 30-turn deterministic sweep selected `compression_policy` as the best measured token-cost setting and `SHORT_TERM_N=30` as the worst. Estimated cost for 10,000 conversations was \$119.0199 best and \$210.3801 worst using the required input/output prices.

Raw records: `reports/hw06/raw/memory_token_records.csv`, `memory_cost_comparison.json`, `memory_cost_summary.json`.
Probe definitions: `memory_probes.yaml`.

Part 4 - File-Based Memory and Consolidation

`AGENT.md` now contains project, technology, coding, testing, and prohibited-action instructions. `MEMORY.md` stores durable facts and includes a timestamp. The `/memory`, `/compact`, `/dream`, and `/help` commands are implemented in `file_memory.py`. The original contradictory 55-line fixture is preserved in `reports/hw06/raw/MEMORY_original.md`.

The actual consolidation reduced the original fixture to 32 nonblank/noncomment facts, merged duplicates, and explicitly recorded the MySQL/MongoDB contradiction for review. Deterministic tests verify command output and contradiction handling.

MongoDB-backed memory is queryable and scalable across sessions but depends on a running database and is less directly inspectable as plain text. File-based memory is highly inspectable and restart-persistent, but it requires consolidation and becomes less scalable across many sessions. MongoDB is better for searchable episodic facts; a curated `MEMORY.md` is better for facts that must remain visible and hard to forget.

Verification and AI Use

Verification output is written by `verify_hw6.py` to `reports/hw06/verification.json`. `AI_USE.md` documents the assistant's role, independent verification, detected problems, and corrective changes. `RUN_LOG.txt` records the implementation and experiment commands.

Required final GitHub push/tag and live-service screenshots are not reported as complete until the missing dependency installation and service checks succeed.